Pengenalan Bahasa C

C-Lanjut, Fork, AWK, SED, Gerp

Oleh: Muhammad Mishbah

Pointer

- Pointer = variabel yang menyimpan ALAMAT memori variabel lain.
- & (ampersand) -> ambil alamat sebuah variabel.
- * (asterisk) -> dereference, akses nilai yang ada di alamat itu.
- Berguna kalau fungsi perlu MENGUBAH variabel milik pemanggil

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 5, y = 10;
    swap(&x, &y);
    printf("x=%d y=%d\n", x, y);
    return 0;
}
```

```
x=10 y=5
```

Array

- Nama array sebenarnya adalah alamat elemen pertamanya.
- Nilai sama dengan &nilai[0] , bisa dipakai langsung sebagai pointer.
- Aritmatika pointer: p + i = alamat elemen ke-i, *(p + i) = nilainya.
- Array & pointer sering dianggap dua sisi mata uang yang sama di C.

```c
#include <stdio.h>

int main() {
    int nilai[5] = {10, 20, 30, 40, 50};
    int *p = nilai;

    for (int i = 0; i < 5; i++) {
        printf("nilai[%d]=%d ", i, nilai[i]);
        printf("(ptr: %d)\n", *(p + i));
    }
    return 0;
}
```

nilai[i] dan *(p+i) selalu sama persis

Fork – Membuat Proses baru

- fork() menduplikasi proses yang sedang berjalan. Saat dipanggil, maka akan ada 2 proses identik.
- Return value tiap proses akan berbeda di masing-masing: 0 di proses child, PID child di proses parent, -1 kalau gagal.
- Kode setelah fork() dijalankan 2 kali, sekali oleh parent, sekali oleh child.

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child: PID %d\n", getpid());
    } else if (pid > 0) {
        printf("Parent: PID %d\n", getpid());
        printf("Child PID: %d\n", pid);
    }
    return 0;
}
```

- Fork()
- Exec(“command”)
- Wait()

Grep, awk, sed

- In Linux/Unix text processing, **grep**, **sed**, and **awk** are essential tools, each excelling in different areas.
- **grep** is best for **fast pattern searching**. It scans input line-by-line and outputs matches.
- **sed** is a **stream editor** for inline text transformations. It can substitute, insert, delete, or select text ranges.
- **awk** is a **full-fledged scripting language** for field-based processing, reporting, and computation

Grep – Pencarian teks dasar

- grep pola file -> cetak baris yang mengandung pola tersebut.
- -i : abaikan besar/kecil huruf (case insensitive).
- -v : kebalikan, baris yang TIDAK mengandung pola.
- -c : cetak jumlah baris yang cocok (bukan isinya).
- -n : sertakan nomor baris.

```
$ grep "ERROR" app.log
ERROR failed to connect
ERROR timeout occurred

$ grep -c "ERROR" app.log
2

$ grep -n "ERROR" app.log
2:ERROR failed to connect
```

```
grep "ERROR" log.txt
grep -v "INFO" log.txt # Invert match
grep -A 2 "ERROR" log.txt # Show 2 lines after match
```

Grep - regex

- ^ -> awal baris $ -> akhir baris
- . -> satu karakter sembarang * -> pengulangan 0+
- [ ] -> salah satu karakter di dalamnya, mis. [0-9]
- -E : extended regex -> aktifkan (a|b), {n}, dst.

```
$ grep "^root" /etc/passwd
root:x:0:0:root:/root:/bin/bash

$ grep -E "^(root|admin)" /etc/passwd
# baris yang mulai "root" ATAU "admin"

$ grep -E "[0-9]{3}" access.log
# baris dengan 3 digit angka berurutan
```

awk

- AWK memproses teks per baris, per kolom (field).
- $1, $2, ... = kolom ke-1, ke-2, dst. $0 = seluruh baris. NF = jumlah kolom, NR = nomor baris.
- Pola dasar: awk 'pattern { action }'.
- -F: ganti pemisah kolom default (spasi) jadi karakter lain.

```
$ echo "alice 25 jakarta" | awk '{print $1, $3}'
alice jakarta

$ awk -F: '{print $1}' /etc/passwd
root
daemon

$ awk '{print NR, $0}' data.txt
1 apple
```

awk

- AWK memiliki variabel & operator matematika bawaan, cocok buat agregasi cepat tanpa perlu membuat program C.
- END { } dijalankan sekali di akhir, setelah semua baris diproses.
- Pattern boleh berupa kondisi angka, sementara baris yang tidak cocok otomatis dilewati.

```
$ awk '{sum += $2} END {print "Total:", sum}' \
    nilai.txt
Total: 240

$ awk '$3 > 100 {print $1}' dataset.txt
alice
carol

# dipakai juga buat filter output ps aux
$ ps aux | \
    awk '$3 > 5.0 {print $2, $11}'
```

sed

- sed = stream editor, transformasi teks baris demi baris tanpa buka editor.
- s/cari/ganti/ -> ganti kemunculan PERTAMA per baris.
- s/cari/ganti/g -> ganti SEMUA kemunculan (global).
- -n '2,4p’ -> cetak baris 2 s.d. 4 saja.
- /pola/d -> hapus baris yang cocok pola.

```
$ cat file.txt
foo apple foo banana

$ sed 's/foo/bar/' file.txt
bar apple foo banana
# cuma foo PERTAMA yang berubah

$ sed '/^#/d' config.txt
# hapus semua baris komentar
```